

Temperature Predictor: An End-to-End IoT and Machine-Learning System for Seven-Day Temperature Forecasts

Konstantinos Giantselidis
Department of Information and Electronic Engineering
International Hellenic University
Thessaloniki, Greece
konstantinosyants@gmail.com

Abstract—Weather forecasts describe grid cells and airport stations, not the balcony where a sensor can be quickly and cheaply stationed. This project, Temperature Predictor, presents an end-to-end system that turns a low-cost sensor into a local seven-day forecast of daily maximum and minimum temperatures. An ESP8266 microcontroller with a DHT22 sensor posts a reading every minute to a FastAPI service, which authenticates the device and stores the reading in a PostgreSQL database. A nightly worker aggregates the readings into daily statistics, then, runs fourteen ridge regression models (one maximum and minimum forecast per target) and stores the results, which finally, a Streamlit dashboard reads through the API. Because the new sensor has no previous history, the ML models were trained on ERA5-Land reanalysis data for Thessaloniki and restricted to the six daily variables the sensor measures. On 616 held-out days, the maximum-temperature of a day model reaches a mean absolute error of 1.62 °C for the next day and 2.74 °C on the seven days forecast. The source code is available at github.com/congiants/Temperature_Predictor.

I. INTRODUCTION

Weather services forecast for grid cells and stations, while a sensor on a specific balcony measures conditions that neither can describe exactly. Low-cost microcontrollers make such measurements cheap. This project asks a practical engineering question: can one low-cost sensor feed a complete, maintainable forecasting system, from the hardware to a dashboard?

Temperature Predictor answers this question with a working pipeline. The sensor posts readings; the API authenticates and stores them; the worker turns them into daily statistics and seven-day forecasts of maximum and minimum temperature; the dashboard shows both. A first version (2024) used XAMPP, PHP, a local MySQL database and a ridge model trained on station data. When its database was lost, the system was rebuilt in 2026 with the goal of implementing production practices: a containerized database, a typed, self-documenting API, device authentication, and a decision log that records all design choices with their rationale and other rejected alternatives, in the style of architecture decision records [1].

The intended application is monitoring and forecasting the microclimate of a single specific place, such as a balcony, a garden, a greenhouse or a rooftop. The project is created with a steady power source in mind. Home owners and gardeners can prepare for frost or heat that the city forecast misses, small

growers can watch a greenhouse, and building managers can follow the conditions around a building. Each site needs only a sensor and WiFi, and one server can serve many sensors. The demonstration deployment is a single sensor on a balcony in Thessaloniki, Greece.

The rebuild had a fixed deadline and was planned with a risk-first mindset. The hardware, the only part that could fail for reasons outside the code, so it was built first, with a software simulator as the fallback in case of project stopping errors. Work that did not serve the end-to-end path was declared as future work.

II. RELATED WORK

Low-cost IoT weather stations are a common design. Kodali and Mandal [2] connect a DHT11, a pressure sensor, a rain sensor and a light sensor to a NodeMCU (ESP8266) board and alert the owner by SMS, e-mail and Twitter when a threshold is crossed. Alam *et al.* [3] combine a NodeMCU and an Arduino with a DHT22, an anemometer, a wind vane and a rain gauge, and publish the readings on the ThingSpeak cloud platform. Kapoor and Barbhuiya [4] use Raspberry Pi nodes that report through a base station to a cloud database, where a Django web application shows the data and the model that is trained on it. Most such systems stop at monitoring and alerting, and several depend on a third-party cloud platform.

Different projects also add forecasting. Yu *et al.* [5] feed dense street-level IoT temperature sensors in New York City to a long short-term memory (LSTM) network that forecasts the next day hour by hour and beats persistence, ARIMA and other baselines; adding five years of station history improved its accuracy. Niculescu *et al.* [6] store the readings of a custom station in SQLite, show them on a Flask web platform and forecast temperature with linear regression, the design closest to this work. Baramatiwala *et al.* [7] send ESP32 readings with LoRa to the cloud and compare regression and gradient-boosting models for the next day temperature.

Operational forecasts are created from numerical weather prediction, which simulates atmospheric physics forward from the current measured state. Data-driven models have recently been becoming competitive: WeatherBench [8] set a common benchmark, and GraphCast [9], a graph neural network trained on decades of reanalysis, outperformed a leading operational

system on most targets up to ten days in the future. Such models are on the global level, and use hundreds of variables. This project sits at the opposite end: it is based on only one location with six variables and a much simpler linear model.

Temperature Predictor differs from the low-cost systems in three ways. It runs its own authenticated API and database instead of a third-party platform; it forecasts seven days of maxima and minima with a model trained on decades of reanalysis data [10]. In line with the finding of Yu *et al.* that long history helps. Finally, it stores every forecast, so that forecasts can later be checked against measurements.

III. REQUIREMENTS

The requirements follow from the application described in Section I. Table I lists the functional requirements, essentially, what the system has to do. Table II describes the technical requirements, the qualities and constraints it must meet. TR5 is specific to this exact project: a new sensor has no past records, so the model has to only use inputs that the sensor provides and must work after one complete day of data.

TABLE I
FUNCTIONAL REQUIREMENTS

FR1	Measure air temperature and relative humidity at the site every 60 s.
FR2	Register a new sensor and issue its credentials.
FR3	Accept readings only from registered, authenticated sensors.
FR4	Store every reading with its timestamp and a quality flag.
FR5	Compute the daily maximum, minimum and mean values for each sensor.
FR6	Every night, automatically forecast the daily maximum and minimum temperature of the next seven days.
FR7	Show recent readings and the latest forecast, with its base date, on a web dashboard.
FR8	Keep past forecasts for later comparison with the measurements.

TABLE II
TECHNICAL REQUIREMENTS

TR1	Security: devices authenticate with tokens; only token hashes are stored; no secrets in version control.
TR2	Data integrity: physical ranges are validated by the API and enforced again by the database.
TR3	Reliability: ingestion is independent of batch jobs; batch jobs can be re-run safely, and one failed night does not stop them.
TR4	Hardware: runs on an ESP8266 (2.4 GHz 802.11b/g/n WiFi, no real-time clock) powered over USB.
TR5	Model inputs: only variables the sensor measures; a forecast must be created after one complete day.
TR6	Maintainability: Python outside the firmware, containerized database, self-documenting API, logged design decisions.
TR7	Accuracy: forecasts must beat the persistence baseline.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the system as six layers. A refinement of the perception, network and application layers commonly used to describe IoT systems [11]. Readings flow upward: the device layer measures, the network layer moves each reading to the service layer, which validates it and hands it to the data layer; the processing layer makes stored readings into daily statistics and forecasts, and finally, the application layer showcases

them. Each layer is designed to be a separate program behind a narrow interface, so each one can be changed or scaled separately.

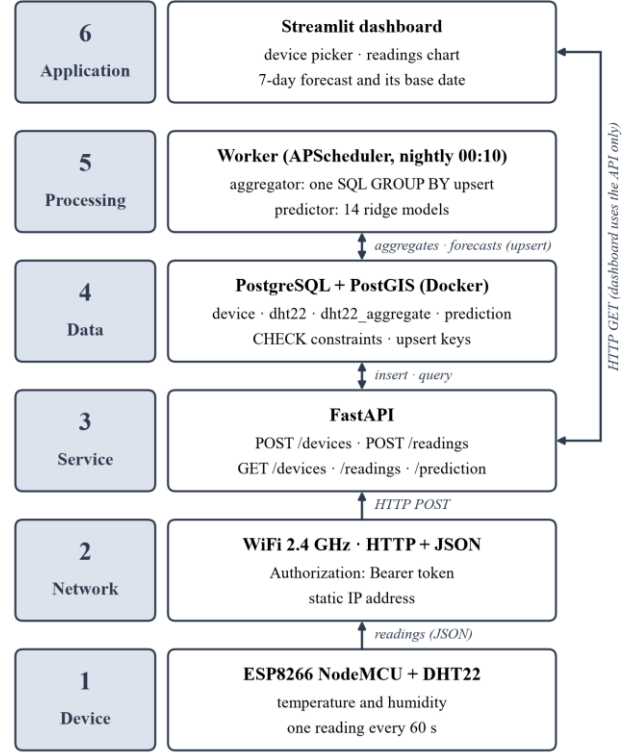


Fig. 1. The six layers of Temperature Predictor. Arrows show the flow of readings and forecasts.

A. Device Layer

An ESP8266 NodeMCU board reads a DHT22 temperature and humidity sensor (Fig. 2) every 60s, or about 1,440 readings a day. The firmware, adapted from the 2024 ESP8266 code, builds a JSON body with the specific device ID, temperature and humidity, and finally, reads the WiFi credentials and device token from a header file that is excluded from version control for safety. The ESP8266 has no reliable internal clock, so timestamps are added by the server. Because the board runs on USB power, it stays awake between each reading. Deep sleep, which would save much more power, but would also need an extra wire to wake up and a new WiFi reconnection cycle, is left for a future battery version of the device.

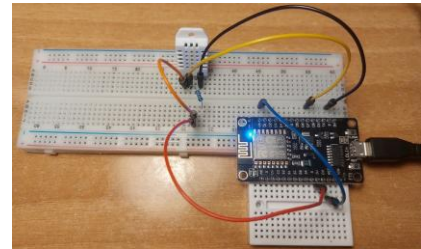


Fig. 2. The sensor node: ESP8266 NodeMCU and DHT22 on breadboards, powered over USB. The blue D0-RST wire in this 2024 photo is needed only for deep sleep, which the current build does not use.

B. Network Layer

Readings travel using 2.4 GHz WiFi as HTTP POST requests. The token in the standard `Authorization: Bearer` header, which keeps it out of URLs and logs. HTTP was chosen for its simplicity, since every tool and framework understands it; publish–subscribe protocols such as MQTT use less bandwidth and energy and suit large fleets better [12]. The hardest practical problem arose at this layer: the ESP8266, limited to 802.11b/g/n, joined a WiFi 6 router but never received a DHCP address, so the firmware now uses a static address for now.

C. Service Layer

A FastAPI application exposes five endpoints: `POST /devices` registers a sensor, `POST /readings` accepts readings, and `GET /devices`, `GET /readings` and `GET /prediction` serve clients. All are automatically documented by OpenAPI. Pydantic schemas make sure every request is valid and kept separate from the SQLAlchemy database models. Internal columns such as the token hash never leave the server for safety. The server generates each device’s random UUID and token. It returns the raw token only once and stores only its SHA-256 hash, so in case of a database leak, no usable credentials can be exposed. Readings from unknown devices are rejected with a 404 code and wrong tokens with a 401 code. Valid readings always receive a server timestamp. Physical ranges (−80 to 80 °C, 0–100 %) are checked here and again by the database. Registration itself is still unauthenticated, which is acceptable for a private network.

D. Data Layer

PostgreSQL with PostGIS run inside a Docker container that holds four tables: `device`, `dht22` for raw readings, `dht22_aggregate` with one row per device and day, and `prediction` with one row per device, base date and target date. Foreign keys, CHECK constraints and indexes on device and time guard the data and keep queries fast. Composite primary keys double as upsert keys, so recomputing a day or a forecast overwrites rows instead of duplicating them. Business logic stays in versioned application code rather than in stored procedures, for version control reasons.

E. Processing Layer

A worker process runs every night at 00:10 Athens time using APScheduler. It is separate from the API so that batch work can never slow ingestion. It first aggregates, with one `INSERT ... SELECT ... GROUP BY ... ON CONFLICT DO UPDATE` statement executed inside PostgreSQL, so raw readings never leave the database. It then predicts: it reads each device’s newest complete day, runs the models of Section V and upserts seven forecast rows per device. Forecasts are stored rather than computed per request, because their inputs change only once a day and a stored history can later be compared with the future measurements. Failures are logged without stopping the scheduler, so both jobs can be re-run safely.

F. Application Layer

A Streamlit dashboard lets the user pick a sensor and shows its recent readings and its latest seven-day forecast, together with the date the forecast is based on (Fig. 3). It reads everything through the API, never directly from the database, so database credentials can live only in a single service.

V. FORECASTING MODEL

A. Training Data

A new sensor never has history, so the model learns from decades of city weather. Version 1 used NOAA GHCN-Daily records from Thessaloniki airport [13], but 32.5 % of the daily maxima and 41.4 % of the minima were missing. Forward-filling them had turned about 40 % of the targets into copies, which probably decreased the reported next-day error of about 1 °C. The rebuild uses reanalysis, which is a combination of a weather model with historical observations and a consistent, gap-free record [10]: ERA5-Land [14] at about 9 km resolution for the Thessaloniki city centre. 28,017 days from 1950 to September 2026. Training uses 1979–2024. 1979 was chosen specifically for the satellite era weather measurements. The columns were renamed to match the sensor’s aggregate table exactly.

B. Model and Evaluation

Every input must be available from the sensor at prediction time, which avoids training–serving skew [15]: the daily maximum, minimum and mean of temperature and humidity. The targets are the maximum and minimum of each of the next seven days, giving fourteen independent ridge regression models [16] ($\alpha = 0.1$), built with scikit-learn [17]. Ridge regression keeps the fit stable when inputs are strongly correlated, as a day’s temperatures are. The v1 recipe also used a 30-day rolling mean, so a new sensor would have waited 30 days for its first forecast; version 2 needs one complete day. The data are split by time, never shuffled [18]: training on 1979–2024 and testing on 616 days from January 2025 to September 2026. Accuracy is measured with the mean absolute error [19]

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (1)$$

where y_i is the observed and $\hat{y}_i$ the predicted value.

The forecasts are tested against simple reference forecasts [20]. Persistence predicts that a future day will equal today. Climatology instead, predicts the long-term average for the date.

C. Results

TABLE III
MEAN ABSOLUTE ERROR (°C) ON THE TEST PERIOD. PERSIST.: PERSISTENCE
BASELINE (DAY N EQUALS TODAY)

Day	Maximum		Minimum	
	Persist.	Ridge v2	Persist.	Ridge v2

1	1.66	1.62	1.38	1.10
2	2.34	2.19	1.89	1.65
3	2.65	2.44	2.23	1.97
4	2.77	2.54	2.39	2.19
5	2.89	2.66	2.60	2.30
6	3.00	2.73	2.75	2.39
7	3.06	2.74	2.82	2.41

Table III compares the models with persistence. Next-day maximum temperature is predicted at a MAE of 1.62 °C, rising to 2.74 °C at day 7 as today’s weather can tell less about the days ahead. The v1 recipe scores the same on day 1 but 2.91 °C on day 7, so removing the cold start cost no accuracy. The next-day minimum reaches 1.10 °C. The model beats persistence in every case, for the maximum by only 0.04 °C on day 1 but by 0.32 °C on day 7, and for the minimum by 0.20–0.41 °C. Climatology is harder to beat: its error stays near 2.38 °C for the maximum and 2.16 °C for the minimum, so it outperforms the model from day 3 and day 4 respectively, because the model knows today’s weather but not the date. End to end, the chain works: the balcony sensor’s first reading was stored on 23 September 2026, and its forecasts now appear on the dashboard (Fig. 3).

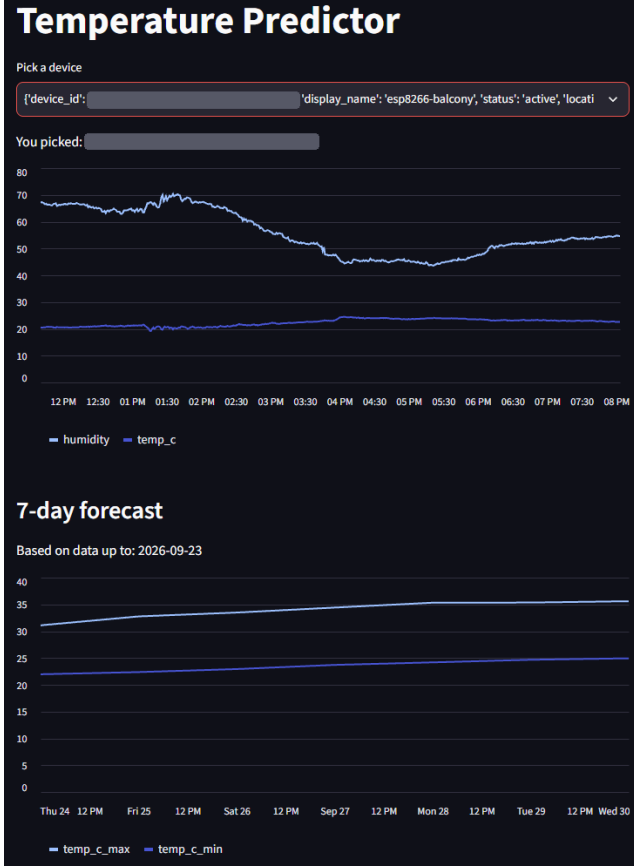


Fig. 3. Streamlit dashboard for the balcony sensor: the device picker (identifier hidden), its last 500 readings and its latest stored forecast. This forecast comes from the incomplete first day of 23 readings and shows the extrapolation discussed in Section VI-D.

VI. CRITICAL EVALUATION

A. Requirements Coverage

All functional requirements in Table I are met, although FR8 cannot be executed until forecasts are old enough to compare with measurements. The technical requirement TR1 of Table II, is only partly met, because tokens travel over plain HTTP and registration is open, due to a lack of time for the assignment. TR7 is met: the model beats persistence at every target (Table III).

B. Scalability

One sensor adds 1,440 rows a day. A thousand sensors would send about 17 requests per second and would add 1.44 million rows a day, or 526 million a year. The API keeps no session state, so it can be replicated behind a load balancer. The only bottleneck is the single database. At that volume the readings table would have to be partitioned by time or changed into a time-series database [21]. Furthermore, raw readings could be discarded after aggregation, because the model needs only daily values. Aggregation already runs inside the database on an index of device and time. The weakest link is operational: the scheduler is just a process on a laptop that needs a container with a restart policy. For thousands of battery-powered sensors, MQTT would probably have to replace one HTTP request per reading [12].

C. Limitations

The hardware is inexpensive, but the project is clearly not a product yet, but a prototype. The aim of the project was to use as much as real production-grade methods and techniques in the purpose of learning given the time restrictions. It lacks TLS and authenticated registration, thus not being safe. Currently, it complies the WiFi credentials into the firmware, has no over-the-air updates, no buffering of readings during network outages and no monitoring. Sensor accuracy needs attention as a DHT22 compared with a laboratory probe measured accurately but responded slowly to temperature changes [22]. Currently, there hasn’t been a complete day with data collection and prediction, due to the assignment time restriction. One area which currently has no clear procedures in place is a case of a board malfunction. Incomplete days must be rejected. Furthermore, the aggregator groups readings by UTC date while training uses Athens days, so the nightly forecast is late by one day. Importantly, a model that takes a single geographical spot into account cannot compete with free numerical forecasts beyond a day or two. Finally, the evaluation uses a single time split, again due to time restrictions for the assignment.

D. Commercial Applicability

The current project is clearly a university project, meant for learning, though aiming to incorporate as much real development practices as possible with the current time restrictions. A commercial service would probably have to combine a local sensor and a professional forecast. It would probably have to take the professional forecast and correct it with local measurements, using the output statistics approach [23]. The sensor

network, authenticated API and stored forecasts built here are the infrastructure that such a service needs though.

VII. CONCLUSION AND FUTURE WORK

Temperature Predictor shows that one low-cost sensor can feed a complete, layered and maintainable forecasting system. It utilized many production grade techniques and was a great source of learning for the engineer in charge.

Future work would have to first focus on TLS, authenticated registration and device provisioning. Then quite non-time taxing and rewarding would be an improvement of the models of the project: day-of-year features, cross-validation and feature selection would be next. Next, containers for every service, time-partitioned storage and rejection of incomplete days would be a next step. Finally sensor calibration, and correction of professional forecasts with local measurements would be an interesting future improvement.

ACKNOWLEDGMENT

Weather data by Open-Meteo.com (CC BY 4.0), containing modified Copernicus Climate Change Service information (2026); station data from NOAA GHCN-Daily.

REFERENCES

- [1] J. Tyree and A. Akerman, "Architecture decisions: Demystifying architecture," *IEEE Softw.*, vol. 22, no. 2, pp. 19–27, Mar. 2005, doi: 10.1109/MS.2005.27.
- [2] R. K. Kodali and S. Mandal, "IoT based weather station," in *Proc. Int. Conf. Control, Instrum., Commun. Comput. Technol. (ICCICCT)*, Kumaracoil, India, Dec. 2016, pp. 680–683, doi: 10.1109/ICCICCT.2016.7988038.
- [3] M. J. Alam *et al.*, "Low cost IoT based weather station for real-time monitoring," in *Proc. IEEE 2nd Int. Conf. Circuits Syst. (ICCS)*, Chengdu, China, Dec. 2020, pp. 127–133, doi: 10.1109/ICCS51219.2020.9336596.
- [4] P. Kapoor and F. A. Barbhuiya, "Cloud based weather station using IoT devices," in *Proc. IEEE Region 10 Conf. (TENCON)*, Kochi, India, Oct. 2019, pp. 2357–2362, doi: 10.1109/TENCON.2019.8929528.
- [5] M. Yu *et al.*, "Using long short-term memory (LSTM) and Internet of Things (IoT) for localized surface temperature forecasting in an urban environment," *IEEE Access*, vol. 9, pp. 137406–137418, 2021, doi: 10.1109/ACCESS.2021.3116809.
- [6] C. A. Niculescu *et al.*, "Real-time environmental parameter monitoring and temperature forecasting using a custom IoT weather station and linear regression," in *Proc. 14th Int. Symp. Adv. Topics Elect. Eng. (ATEE)*, Bucharest, Romania, Oct. 2025, pp. 1–5, doi: 10.1109/ATEE66006.2025.11299969.
- [7] I. Baramatiwala *et al.*, "IoT-based microclimate monitoring and temperature forecasting using machine learning," in *Proc. 4th Int. Conf. Sustain. Comput. Smart Syst. (ICSCSS)*, Coimbatore, India, Jul. 2026, pp. 312–317, doi: 10.1109/ICSCSS69635.2026.11646138.
- [8] S. Rasp, P. D. Dueben, S. Scher, J. A. Weyn, S. Mouatadid, and N. Thuerey, "WeatherBench: A benchmark data set for data-driven weather forecasting," *J. Adv. Model. Earth Syst.*, vol. 12, no. 11, Art. no. e2020MS002203, Nov. 2020, doi: 10.1029/2020MS002203.
- [9] R. Lam *et al.*, "Learning skillful medium-range global weather forecasting," *Science*, vol. 382, no. 6677, pp. 1416–1421, Dec. 2023, doi: 10.1126/science.adi2336.
- [10] H. Hersbach *et al.*, "The ERA5 global reanalysis," *Q. J. R. Meteorol. Soc.*, vol. 146, no. 730, pp. 1999–2049, Jul. 2020, doi: 10.1002/qj.3803.
- [11] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347–2376, 2015, doi: 10.1109/COMST.2015.2444095.
- [12] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *Proc. IEEE Int. Syst. Eng. Symp. (ISSE)*, Vienna, Austria, Oct. 2017, pp. 1–7, doi: 10.1109/SysEng.2017.8088251.
- [13] M. J. Menne, I. Durre, R. S. Vose, B. E. Gleason, and T. G. Houston, "An overview of the Global Historical Climatology Network-Daily database," *J. Atmos. Ocean. Technol.*, vol. 29, no. 7, pp. 897–910, Jul. 2012, doi: 10.1175/JTECH-D-11-00103.1.
- [14] J. Muñoz-Sabater *et al.*, "ERA5-Land: A state-of-the-art global reanalysis dataset for land applications," *Earth Syst. Sci. Data*, vol. 13, no. 9, pp. 4349–4383, Sep. 2021, doi: 10.5194/essd-13-4349-2021.
- [15] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Boston, MA, USA, Dec. 2017, pp. 1123–1132, doi: 10.1109/BigData.2017.8258038.
- [16] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, Feb. 1970, doi: 10.1080/00401706.1970.10488634.
- [17] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, no. 85, pp. 2825–2830, 2011.
- [18] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *Int. J. Forecast.*, vol. 16, no. 4, pp. 437–450, Oct. 2000, doi: 10.1016/S0169-2070(00)00065-0.
- [19] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance," *Clim. Res.*, vol. 30, pp. 79–82, 2005, doi: 10.3354/cr030079.
- [20] A. H. Murphy, "What is a good forecast? An essay on the nature of goodness in weather forecasting," *Weather Forecast.*, vol. 8, no. 2, pp. 281–293, Jun. 1993, doi: 10.1175/1520-0434(1993)008<0281:WIAGFA>2.0.CO;2.
- [21] Y. Hao *et al.*, "TS-Benchmark: A benchmark for time series databases," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE)*, Chania, Greece, Apr. 2021, pp. 588–599, doi: 10.1109/ICDE51399.2021.00057.
- [22] Y. A. Ahmad *et al.*, "On the evaluation of DHT22 temperature sensor for IoT application," in *Proc. 8th Int. Conf. Comput. Commun. Eng. (ICCCCE)*, Kuala Lumpur, Malaysia, Jun. 2021, pp. 131–134, doi: 10.1109/ICCCCE50029.2021.9467147.
- [23] H. R. Glahn and D. A. Lowry, "The use of model output statistics (MOS) in objective weather forecasting," *J. Appl. Meteorol.*, vol. 11, no. 8, pp. 1203–1211, Dec. 1972, doi: 10.1175/1520-0450(1972)011<1203:TUOMOS>2.0.CO;2.
- [24] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 28, 2015, pp. 2503–2511.